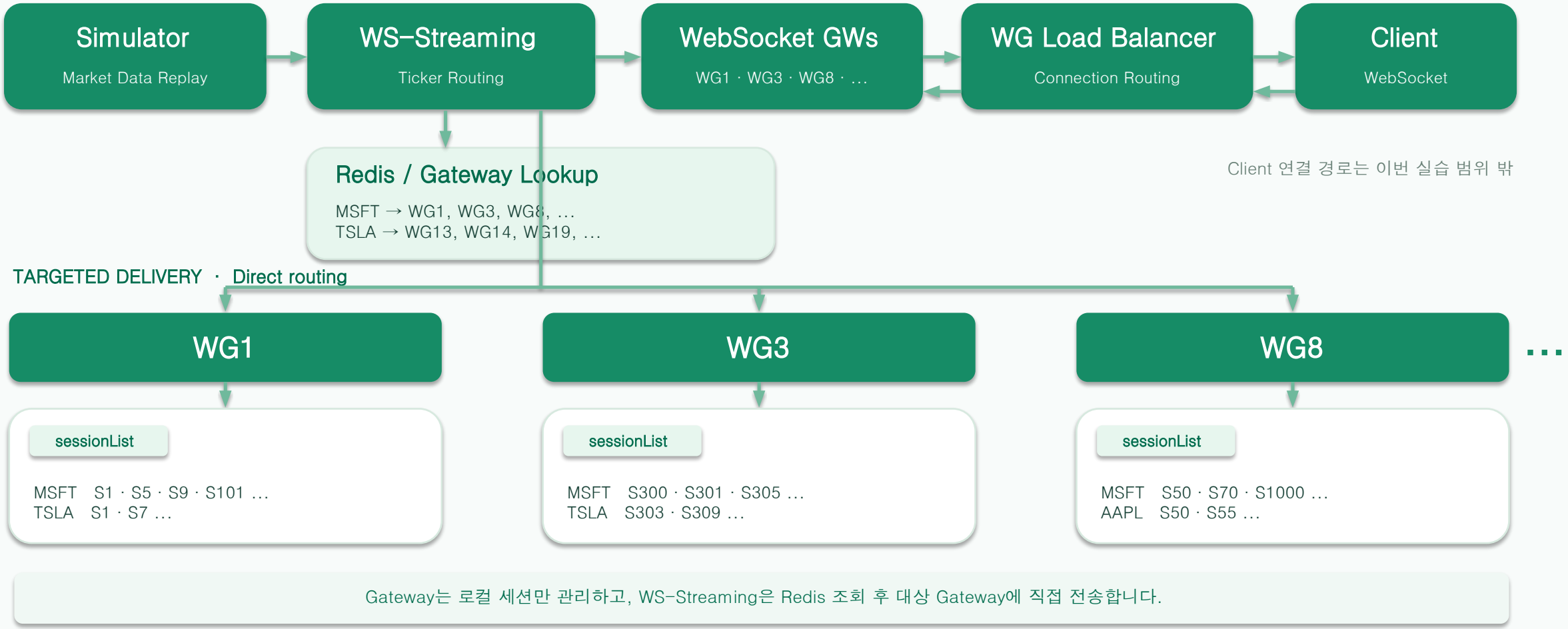
Market Data Streaming | Gateway Routing

DIRECT

시세 이벤트 직접 전달과 Gateway별 세션 관리를 하나의 다이어그램으로 표현합니다.

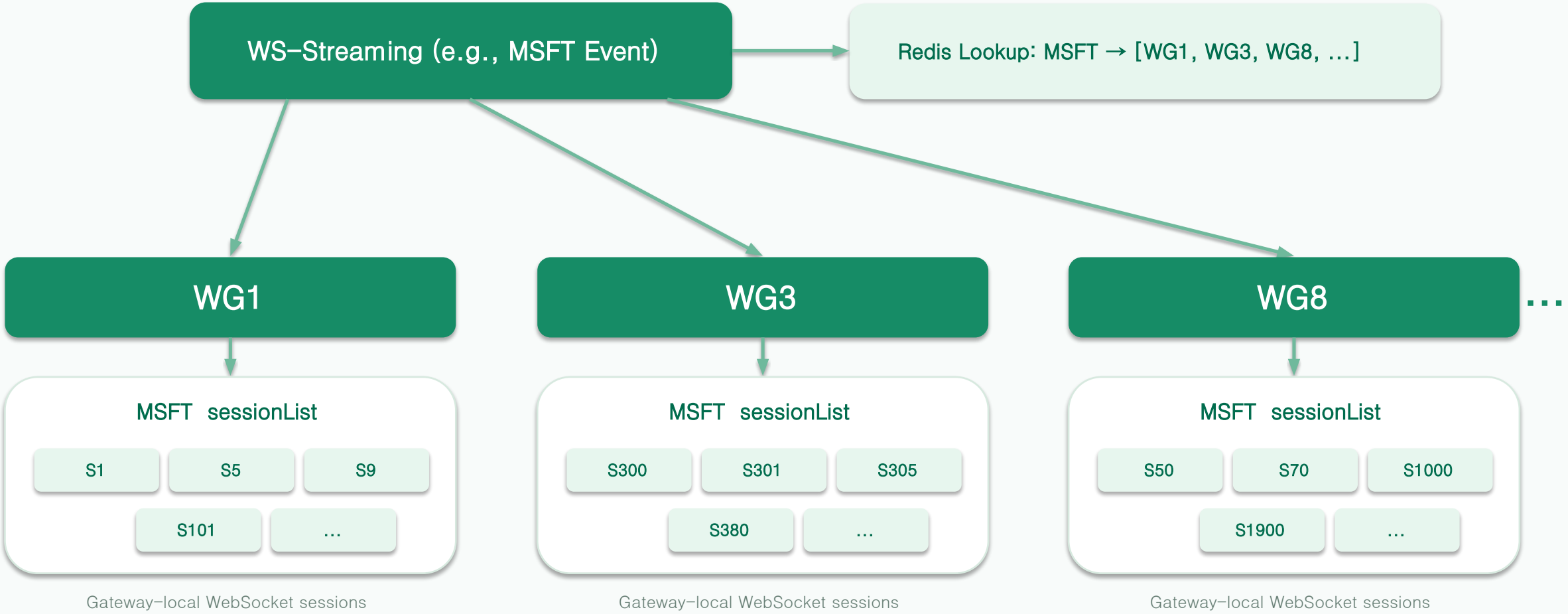


Gateway는 로컬 세션만 관리하고, WS-Streaming은 Redis 조회 후 대상 Gateway에 직접 전송합니다.

Session Fan-out by Gateway | Example: MSFT

FAN-OUT

예시로 MSFT 이벤트를 WG1·WG3·WG8에 각각 전달한 뒤 로컬 Session List로 분배합니다.

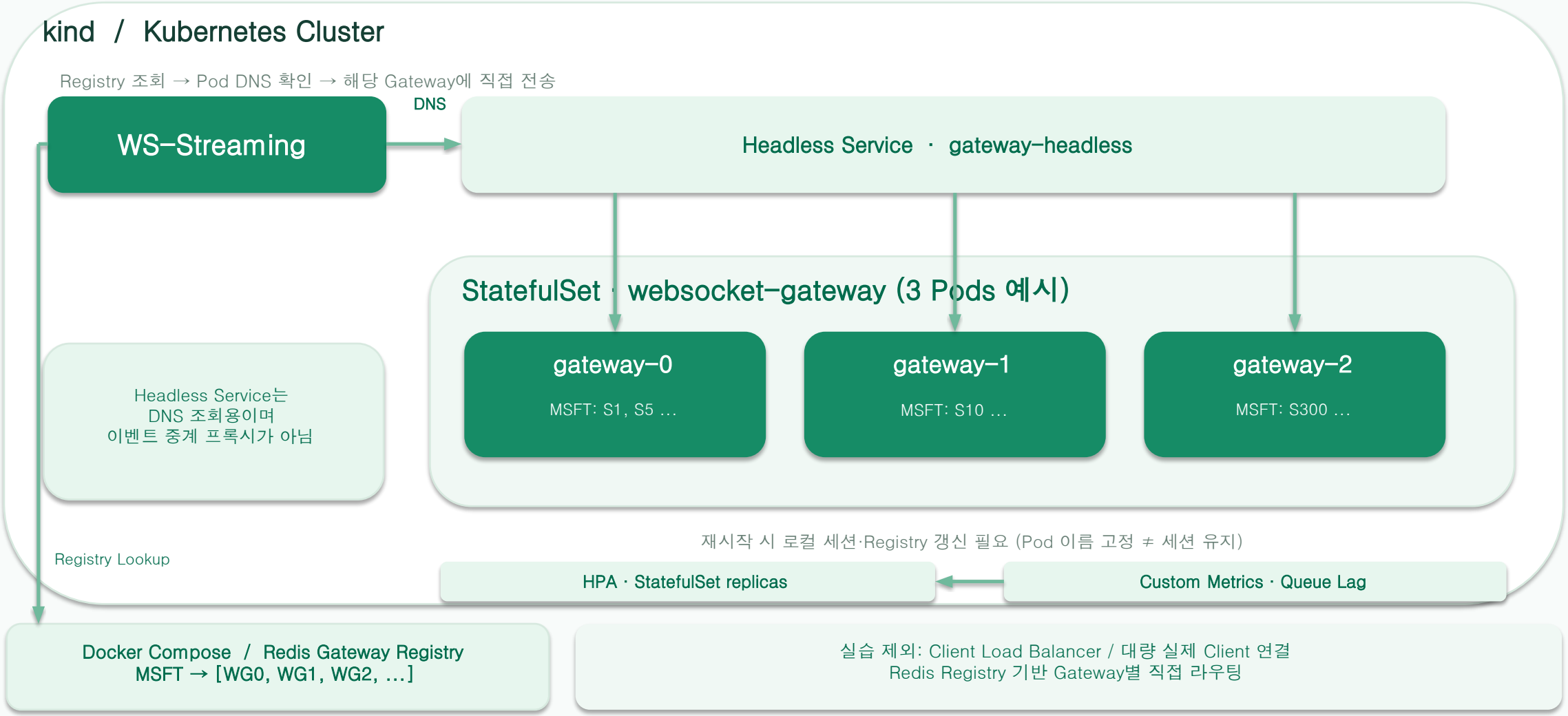


Gateway 목록 조회 후 직접 라우팅 · 세션 목록은 각 Gateway 로컬에서 관리

WebSocket Gateway | Kubernetes Architecture

K8S

WS-Streaming과 Gateway는 kind 내부 · Headless Service로 Pod별 DNS 조회 후 직접 전송



Scale-out | Synthetic 1ms Send Constraint

SCALING

다종목 시세 Replay · Gateway별 1ms 직렬 송신 제약 · HPA 3~20 Replica Scale-out

1ms / session send
Gateway당 단일 직렬 송신 큐 가정

1,000 sends/s
Gateway 1개 이론적 용량

3,000–20,000 sends/s
HPA 3~20개 Replica · 합산 이론적 용량

Scale-out 실행 흐름

① Simulator
주식 시세 이벤트 증가

→

② Gateway Queue
대기 작업 적체

→

③ HPA
Custom Metrics 감지

→

④ New Gateway
신규 가상 세션 배치

Grafana / HPA 지표

gateway_pending_send_jobs · queue_wait_p99
active_virtual_sessions · processed_sends/s

중요한 제약

HPA는 Pod만 추가합니다. 기존 가상 세션을 신규 Gateway로 재배치해야 총 처리량이 증가합니다.

HPA 범위: 3~20 Replicas | Gateway당 1ms 직렬 송신 제약을 둔 이론적 시뮬레이션
실제 WebSocket TPS 측정이 아니며 비동기 sleep(1) 병렬 실행은 처리량 제한을 재현하지 못합니다.

Gateway Pod Termination | Redis Registry Cleanup

LIFECYCLE

Gateway 정상 종료 시 즉시 정리하고, 비정상 종료 시에는 Heartbeat와 주기적 Sweeper로 오래된 등록 정보를 정리합니다.

A. 정상 종료 · SIGTERM / Scale-down

① Pod 종료 시작
gateway-1 / SIGTERM

→

② 신규 작업 중지
Readiness = false

→

③ Redis Registry 정리
구독 종목 목록에서 gateway-1 제거

→

④ 종료 완료
연결·리소스 정리

Before · Redis Set: ticker:MSFT:gateways
{gateway-0, gateway-1, gateway-2}

→

After · gateway-1 멤버십 제거
{gateway-0, gateway-2}

B. 비정상 종료 · Crash / Node Failure

Pod가 정리 코드 실행 없이 종료
Redis Set에는 stale entry 잔존

→

Heartbeat 만료 여부 확인
주기적 Registry Sweeper

→

오래된 Registry 멤버 제거
현재 등록 상태 재검증 후 SREM

Heartbeat TTL 만료는 정리 후보 신호입니다. 일시적인 Redis 장애도 고려해 재검증한 뒤 정리합니다.
Gateway ID 및 재시작 세대 식별 방식은 추후 결정합니다. Redis Set 멤버에는 개별 TTL이 없습니다.

Gateway Heartbeat | TTL-based Failure Detection

HEARTBEAT

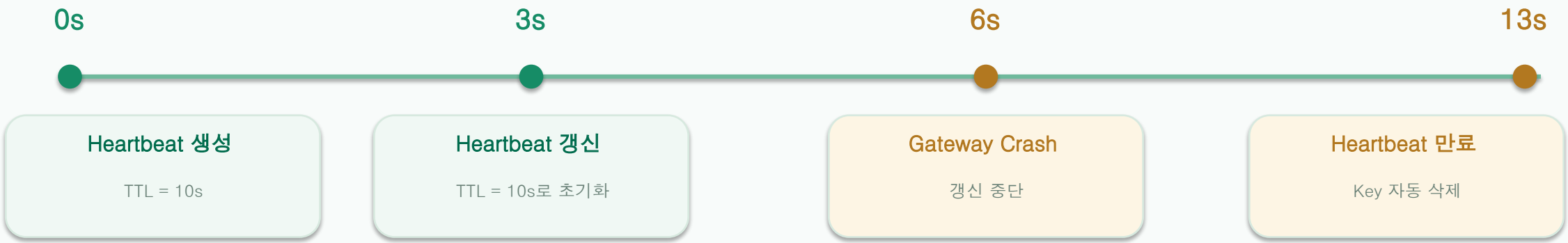
Gateway가 Redis에 주기적으로 생존 신호를 갱신하고, 갱신이 중단되면 TTL 만료로 장애 후보를 식별합니다.

3초
Heartbeat 갱신 주기

10초
Redis Key TTL

13초
시나리오 내 만료 시각

WG3 장애 감지 타임라인 (예시)



Redis: gateway:WG3:heartbeat

SET key <value> EX 10 · 3초마다 TTL 갱신

TTL 만료 ≠ Gateway Set 자동 정리

Sweeper가 생존 상태 재확인 후 멤버 제거

6초 Crash 이후 마지막 갱신(3초)으로부터 10초가 지나 13초에 만료됩니다. 만료는 장애 확정이 아닌 정리 후보 신호입니다.

Redis Gateway Registry | Data Structures & Periodic Cleanup

REDIS

종목별 Gateway 조회, Gateway 생존 확인, 역방향 구독 조회를 분리합니다.

Redis Key / Value 설계

SET · 종목별 대상

ticker:MSFT:gateways

{WG1, WG3, WG8, …}

STRING + TTL · 생존 신호

gateway:WG3:heartbeat

<instanceToken> (예: EX 10초)

SET · 역방향 구독 인덱스

gateway:WG3:tickers

{MSFT, TSLA, …}

추후 결정: Gateway ID / 재시작 instanceToken

Pod 이름 사용 가능성은 있으나 현재 확정하지 않음

Periodic Registry Sweeper

① 주기적으로 등록 Gateway 탐색

등록 인덱스 또는 종목별 Set 조회

↓

② Heartbeat TTL 만료 후보 확인

생존 상태 · 등록 세대 재검증

↓

③ 오래된 Gateway 멤버 정리

관련 ticker Set에서 SREM

↓

④ Registry 정합성 유지

역방향 tickers 인덱스도 정리

시세 전송 경로에서는 Registry를 조회해 직접 라우팅 · Cleanup은 별도 주기적 작업

Heartbeat Key는 TTL로 만료되지만, ticker:*:gateways Set의 멤버는 자동으로 삭제되지 않습니다.

WebSocket Session Management | Pool & Scale-in

CONCEPT ONLY

운영 환경에서 고려할 확장 아이디어 · 이번 실습에서는 구현하지 않음

① 트래픽 급증 구간의 Gateway Pool 분리 (선택적 운영 전략)

Regular Gateway Pool

평상시 신규 연결 수용 · 기본 StatefulSet
예: gateway-regular-0, gateway-regular-1 …

Burst Gateway Pool

개장·마감 전후 급증한 신규 연결 수용
별도 StatefulSet · 시간대별 확장 정책

※ Pool 분리에는 연결 배정 정책이 필요하며, StatefulSet Pod 이름의 접두사만으로 자동 분리되지는 않습니다.

② 사용자 감소 시 Scale-in / Session Consolidation

01 Draining
축소 대상 신규 연결 차단

→

02 Reconnect
다른 Gateway로 재연결

→

03 Resubscribe
관심 종목 구독 복원

→

04 Terminate
세션 정리 후 Pod 종료

실습 범위: HPA Scale-out + 가상 세션 배치만 구현 | 실제 Pool 운영·재연결·Scale-in은 개념 설명

WebSocket Session Lifecycle | Production Considerations

OUT OF SCOPE

실제 WebSocket 연결을 운영할 때 고려할 생존 확인·유휴 정리·송신 보호 정책

Ping / Pong Heartbeat

죽은 TCP/WebSocket 연결 감지
Pong 미응답 시 연결 종료 및 리소스 회수

Idle Timeout

서비스에서 정의한 유휴 조건에 따라 정리
사용자 입력 없음 ≠ 유휴: 시세 수신은 정상 활동

Subscription Lifecycle

구독 종목이 없는 연결·만료된 구독 관리
구독 해제 시 로컬 Session Index 갱신

Slow Consumer / Backpressure

송신 버퍼 누적량·전송 지연 감시
상한 초과 시 최신값 병합·제한·연결 종료 검토

핵심 구분 Gateway Heartbeat(서버 생존)와 Client Ping/Pong(연결 생존)은 서로 다른 계층의 상태 확인입니다.

이번 실습에서는 실제 Client 세션 수명주기를 구현하지 않고, Scale-out 아키텍처와 메트릭 검증에 집중합니다.